Desarrollo de Aplicaciones Web

JavaScript. Sintáxis Básica.

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces

JavaScript. Sintáxis básica.

- Es sensible al uso de mayúsculas/minúsculas
- Es conveniente que todas las líneas de código finalizan en “;”.
- Los comentarios se hacen de dos formas:
  - “//” Para comentar líneas individuales.
  - “/* ….*/” Para comentar grupos de líneas.

JavaScript. Declaración de variables.

var mivariable;

El tipo de variable es el que toma al inicializarse.

Ej.-

```
var mivariable= “pepe”;
var mivariable= 135;
var mivariable= 135.0;
```

Nota.- *con la llegada de la versión ES6, var ha sido sustituido por let y/o const.*

JavaScript. Ámbito de las variables.

- Global.- accesible desde cualquier punto de la página.

```
<SCRIPT>
        var variableGlobal
</SCRIPT>
```

- Local.- accesible sólo dentro de la función.

```
<SCRIPT>
        function miFuncion (){
                var variableLocal
        }
</SCRIPT>
```

JavaScript. Tipos de datos.

- Existen cinco tipos de datos:
  - Cadenas de texto (string).
  - Valores numéricos (number).
  - Booleanos (boolean)
  - Objetos (Object)
  - Nulos (Null)

typeof(*mivariable*).- devuelve el tipo de variable.

JavaScript. Conversión entre tipos.

- Implícita.- la coerción es automática hacia cadenas.
- Explícita.- mediante funciones del tipo:
  - parseFloat()
  - parseInt()
  - toString()

JavaScript. Operadores.

- Aritméticos.- +, -, *, /, ++, - -, %.
- Lógicos.- AND, OR, NOT.
- Relacionales.- ==, !=, >, >=, <, <=, ===, !==.
- Bit a bit.- &, |, ^, ~, <<, >>, >>>.
- Asignación.- =, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=, >>>=.
- Otros operadores.- + (concatenación), ?:, .

JavaScript. Operaciones matemáticas.

En general forman parte del objeto Math.

```
Ej.- resultado= Math.cos(37);
```

Utilizando with, podemos predeterminar el objeto. Desaconsejado en la actualidad.

```
Ej.- with(Math);
     resultado= cos(37);
```

JavaScript. Operaciones matemáticas.

Métodos incluidos en Math:

- log();
- exp();
- sqrt();
- pow();
- abs();
- floor();
- ceil();
- round();
- randon ();
- sin();
- cos();
- tan();
- asin();
- acos();
- atan();
- max();
- min();

JavaScript. Constantes predefinidas.

- E.- cte de Euler.
- LN2.- neperiano de 2.
- LN10.- neperiano de 10.
- LOG2.- $\log_2(e)$.
- LOG10.- $\log_{10}(e)$.
- PI.- número π.
- SQRT2.- raíz cuadrada de 2.
- SQRT1_2.- inversa de raíz cuadrada de 2.

JavaScript. Estructuras de control.

if - else

```
if (condición)
{
        _____
        _____
}
else
{
        _____
        _____
}
```

JavaScript. Estructuras de control.

for

```
for (valor inicial; condición parada; incremento)
{
        _____
        _____
}
```

for-in

```
for (dato in Objeto.valor)
{
        _____
        _____
}
```

JavaScript. Estructuras de control.

while

```
while (condición)
{
        _____
        _____
}
```

do while

```
do
{
        _____
        _____
}
while (condición);
```

JavaScript. Estructuras de control.

Salida de bucles:

- break; salida sin cumplir la condición.
- continue; ejecuta la siguiente vuelta, sin llegar al final del código.

JavaScript. Estructuras de control.

Switch-case

```
switch (expresión)
{
        case caso_1:
                _________
                _________
                break;
        case caso_2:
                _________
                _________
                break;
        .
        .
        .
        case caso_n:
                _________
                _________
                break;
}
```

JavaScript. Definición de funciones.

Se definen con la palabra *function.*

En el caso básico, la función se define y se puede llamar posteriormente.

function *mi_función* (*parametros*)
{
_____
_____
return( );
}

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada a la función: <b id="uno"></b></p>
```

```
let x = myFunction(4, 3);
document.getElementById("uno").innerHTML = x;

function myFunction(a, b) {
  c=0;
  if (a>0)
    c= a+b;
  else
    c=a-b;
  return c;
}
```

Funciones JavaScript

Resultado de la llamada a la función: 7

JavaScript. Definición de funciones.

Se definen con la palabra *function.*

Las funciones pueden ser definidas usando expresiones y almacenarse en variables. En este caso la función es anónima.

x= *function*(*parametros*)
{
_____
_____
return( );
}

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada a la función: <b id="uno"></b></p>
```

```
x= function myFunction(a, b) {
  c=0;
  if (a>0)
    c= a+b;
  else
    c=a-b;
  return c;
}
document.getElementById("uno").innerHTML = x(3, 4);
```

Funciones JavaScript

Resultado de la llamada a la función: 7

JavaScript. Definición de funciones.

Se definen con la palabra *function.*

Las funciones pueden autoinvocarse. No llevan return.

(*function*(*parameters*)
{
_____
_____

}) (*valor_parametros*)

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada a la función: <b id="uno"></b></p>
```

```
(function(a, b) {
  c=0;
  if (a>0)
    c= a+b;
  else
    c=a-b;
  document.getElementById("uno").innerHTML = c
})(3, 4);
```

Funciones JavaScript

Resultado de la llamada a la función: 7

JavaScript. Definición de funciones.

Callback→ función pasada como argumento de otra función y ejecutada por esta última.

```
<h2>No Callback</h2>
<p></p>
```

No Callback

20

Opción 1

```
function miCalculo(x1, x2) {
    return x1*x2;
}

function miVisualizacion(dato) {
    document.getElementsByTagName("p")[0].innerHTML= dato;
}

let resultado= miCalculo(5, 4);
miVisualizacion(resultado);
```

Opción 2

```
function miCalculo(x1, x2) {
    let resultado= x1*x2;
    miVisualizacion(resultado);
}

function miVisualizacion(dato) {
    document.getElementsByTagName("p")[0].innerHTML= dato;
}

miCalculo(5, 4);
```

JavaScript. Definición de funciones.

Callback→ función pasada como argumento de otra función y ejecutada por esta última.

```
<h2>Callback</h2>
<p></p>
```

Callback

20

```
function miVisualizacion(dato) {
  document.getElementsByTagName("p")[0].innerHTML= dato;
}

function miCalculo(x1, x2, miFuncionCallback) {
  let resultado= x1 * x2;
  miFuncionCallback(resultado);
}

miCalculo(5, 4, miVisualizacion);
```

JavaScript. Definición de funciones.

Variables locales y globales:

```
<h3>Funciones JavaScript</h3>

<p id="p1"></p>
<p id="p2"></p>
<p id="p3"></p>
```

```
let g= 10;
let a= 4;
let b= 3;

let x= miF_1(a, b);
document.getElementById("p1").innerHTML= "a= " + a;
document.getElementById("p2").innerHTML= "g= " + g;
document.getElementById("p3").innerHTML= "x= " + x;
```

```
function miF_1(a, b) {
  a+= 1;
  g= a+b;
  return a*b;
}
```

Funciones JavaScript

a= 4

g= 8

x= 15

JavaScript. Definición de funciones.

Variables locales y globales:

Llamadas por valor y por referencia → Tipos primitivos (números cadenas, … por valor. Objetos (incluyendo arrays y demás) por referencia.

```
<h3>Funciones JavaScript</h3>

<p id="p1"></p>
<p id="p2"></p>
<p id="p3"></p>
```

```
let a= 4;
let b= new Array (1, 3, 6);

let x= miF_1(a, b);
document.getElementById("p1").innerHTML= "a= " + a;
document.getElementById("p2").innerHTML= "b= " + b;
document.getElementById("p3").innerHTML= "x= " + x;
```

```
function miF_1(a, b) {
  a+= 1;
  b[0]= 2*a;
  return a*b[0];
}
```

Funciones JavaScript

a= 4

b= 10,3,6

x= 50

JavaScript. Objetos.

Tradicionalmente la instanciación de objetos se realizaba mediante la palabra clave new, que invocaría una función que hace de constructor.

```
function MiClase (atr_1, atr_2, …, atr_n){
  this.atr_1= atr_1;
  this.atr_2= atr_2;
  …
  this.atr_n= atr_n;}

miObjeto= new MiClase(par_1, par_2, …, par_n);
```

Los atributos de la clase se llaman propiedades. Su acceso se realiza bien a través del operador “.”, bien a través de “[ ]”

```
miObjeto.atr_1;

miObjeto[“atr_1”]
```

JavaScript. Objetos.

La tendencia actual es usar “{ }”, para instanciar directamente el objeto .

```
miObjeto= {atr_1:v_Atr_1, atr_2:vAtr_2, …, atr_n:vAtr_n}
```

Las propiedades del objeto son públicas y pueden ser accedidos y modificados directamente.

```
miObjeto.atr_1= nuevoValor;
```

Las propiedades pueden ser también eliminadas mediante la función *delete().*

```
delete(miObjeto.atr_1);
```

JavaScript. Objetos.

Cuando instanciamos objetos mediante "{ }" e posible añadir métodos mediante:

```
miObjeto= {atr_1:v_Atr_1,
          atr_2:vAtr_2,
          …,
          miF: function(){
               ……
               return ….;
               }
          };
```

O posteriormente, mediante:

```
miObjeto= {atr_1:v_Atr_1,
          atr_2:vAtr_2,
          …,
          };

miObjeto.miF= function(){
              ……
              return ….;
              }
```

La llamada, en cualquier caso, se realiza mediante el operador .

```
miObjeto.miF()
```

JavaScript. Objetos.

Las funciones son objetos en javascript → Tienen propiedades y métodos.

arguments → Array que almacena los argumentos

arguments.length → Retorna el número de argumentos

toString()→ Método que retorna la función como una cadena

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada a la función: <b id="uno"></b></p>
```

```
let x = miFuncion(4, 3);
document.getElementById("uno").innerHTML = x;

function miFuncion(a, b) {
  return arguments[1]*arguments.length;
}
```

Funciones JavaScript

Resultado de la llamada a la función: **6**

JavaScript. Objetos.

Las funciones son objetos en javascript → Tienen propiedades y métodos.

arguments → Array que almacena los argumentos

arguments.length → Retorna el número de argumentos

toString()→ Método que retorna la función como una cadena

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada a la función: <b id="uno"></b></p>
```

```
let x = miFuncion(4, 3);
document.getElementById("uno").innerHTML = miFuncion.toString();

function miFuncion(a, b) {
  return a+b;
}
```

Funciones JavaScript

Resultado de la llamada a la función: **function miFuncion(a, b) { return a+b; }**

JavaScript. Arrays.

Se crean mediante new. Constituyen un objeto, con métodos propios y propiedades.

Ej.-
```
let proveedor= new Array();
proveedor[0]= "Pepa";
proveedor[1]= "Antonio";
```

Actualmente la forma recomendada es mediante [ ]

Ej.-
```
let proveedor= [ ];
proveedor[0]= "Pepa";
proveedor[1]= "Antonio";
```

O directamente:

```
let proveedor= ["Pepa", "Antonio"];
```

JavaScript. Arrays.

Actualmente la forma recomendada es mediante [ ]

Propiedades y métodos:

- .length;
- .join();
- .reverse();
- .sort();
- .concat();
- .slice();

JavaScript. Manejo de cadenas.

Se realiza mediante el objeto string. La forma recomendada para instanciarlas no es mediante new, sino directamente:

```
let miCadena= "Hola Mundo"
```

Métodos:

- .fromCharCode();
- .charCodeAt();
- .indexOf();
- .lastIndexOf();
- .substr();
- .slice();
- .substring();
- .toLowerCase();
- .toUpperCase();

JavaScript. Programación orientada a eventos.

Eventos:

- Click.- pulsar con ratón
- MouseOver.- posicionarse encima con ratón
- MouseOut.- dejar de señalar con ratón
- Load.- cargar página
- Unload.- abandonar página
- Focus.- ventana pasa a activa
- Blur.- ventana deja de ser activa
- KeyPress.- presionar tecla
- Select.- seleccionar
- Change.- modificar
- Submit.- enviar formulario
- Reset.- inicializa formulario
- Error.- se produce un error
- Abort.- se aborta

JavaScript. Programación orientada a eventos.

Objeto	Evento
window	load, unload, blur, focus
link	click, mouseOut, mouseOver
area	mouseOut, mouseOver
image	abort, error, load
form	reset, submit
test, textarea, password	blur, change, focus, select
button, reset, submit, radio checkbox	onclick
select	blur, change, focus
Controles selección archivos	blur, focus, select

JavaScript. Fechas y Horas.

Se realiza mediante el objeto date.

Dos constructores:

- new Date(“mes día, año horas:minutos:segundos”
- new Date(año, mes, día, [horas, [minutos, [segundos, [milisegundos]]]])

Métodos:

- .getTime();
- .getDay();
- .getDate();
- .getMonth();
- .getYear();
- .getFullYear();
- .getHours();
- .getMinutes();
- .getSeconds();
- .getMiliseconds();
- .setTime();
- .setDay();
- .setDate();
- .setMonth();
- .setYear();
- .setFullYear();
- .setHours();
- .setMinutes();
- .setSeconds();
- .setMiliseconds();

JavaScript. Fechas y Horas.

Hora Universal Coordinada (UTC).- es la denomida hora GMT (Greenwich Meridiam Time). Utiliza métodos análogos a los anteriores +UTC.

Métodos:

- .getUTCTime();
- .getUTCDay();
- .getUTCDate();
- .getUTCMonth();
- .getUTCFullYear();
- .getUTCHours();
- .getUTCMinutes();
- .getUTCSeconds();
- .getUTCMiliseconds();
- .setUTCTime();
- .setUTCDay();
- .setUTCDate();
- .setUTCMonth();
- .setUTCYear();
- .setUTCFullYear();
- .setUTCHours();
- .setUTCMinutes();
- .setUTCSeconds();
- .setUTCMiliseconds();

Conversiones:

- .toUTCString
- .toGMTString

Desarrollo de Aplicaciones Web

JavaScript. Sintáxis Básica.

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces